Software Development Management Lab · Lab 02

Product Backlog & User Stories

From your Project Charter to a prioritized list of requirements

Ekramul Haque Amin
Lecturer, AI and Data Science
Dhaka International University (DIU)

Course Title: Software Development Management | Course No: 0613-307

Today’s Lab Plan

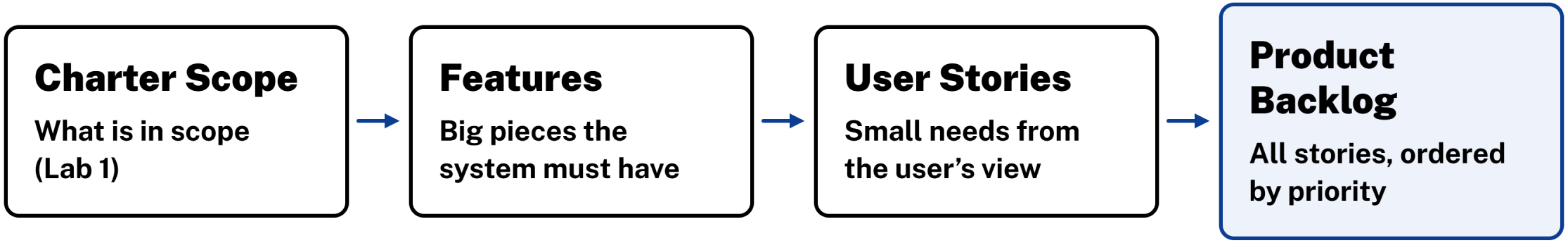
1	Check-in: show your charter’s scope
2	Functional and non-functional requirements
3	Writing user stories and the INVEST checklist
4	Acceptance criteria and MoSCoW prioritization
5	Hands-on: build your product backlog
6	Peer review and wrap-up

Learning Objectives

By the end of this lab, you will be able to:

1. **Tell functional and non-functional requirements apart**
2. **Write user stories in the standard format**
3. **Check stories with the INVEST criteria**
4. **Write acceptance criteria using Given / When / Then**
5. **Prioritize a product backlog with MoSCoW**

From Charter to Backlog



Every story should trace back to something in your charter's scope.

Functional vs. Non-Functional Requirements

Functional

WHAT the system does

- Customer can add items to a cart
- Staff can update the menu
- Customer can pay online

Non-Functional

HOW WELL it does it

- Pages load in under 2 seconds
- Payment data is encrypted
- Works on mobile and desktop

Functional needs become user stories. Non-functional needs are listed separately.

Common Non-Functional Requirements

Performance
Search results appear within 1 second

Security
Passwords are stored encrypted

Usability
A new user can place an order in under 3 minutes

Availability
The system is up 99% of the time

Scalability
Supports 1,000 users at the same time

Compatibility
Works on Android, iOS and Chrome

> **Make them measurable: “fast” is weak, “under 2 seconds” is testable.**

What Is a User Story?

A short, simple description of a feature, written from the user's point of view.

> **As a [type of user], I want [goal] so that [benefit].**

WHO	WHAT	WHY
The user or role	What they want to do	The value they get

Examples: Online Food Ordering

As a customer, I want to browse the menu by category so that I can quickly find what I want to eat.

As a customer, I want to track my order status so that I know when my food will arrive.

As a branch manager, I want to mark a dish as sold out so that customers cannot order it.

Notice: different users (customer, manager), each with a clear benefit.

Weak vs. Better User Stories

WEAK: “The system should have a database.”	BETTER: “As a manager, I want to see daily sales so that I can track income.”
WEAK: “As a user, I want the app to be good.”	BETTER: “As a customer, I want to reorder a past meal in one tap so that ordering is faster.”
WEAK: “As a customer, I want to order, pay, track and rate food.”	BETTER: Split it into four separate stories, one goal each.

INVEST: A Checklist for Good Stories

I	**Independent**	Can be built without waiting for other stories
N	**Negotiable**	Details can be discussed with the customer
V	**Valuable**	Gives real value to a user
E	**Estimable**	The team can estimate how big it is
S	**Small**	Fits inside one sprint
T	**Testable**	Has clear acceptance criteria

Acceptance Criteria: When Is a Story “Done”?

> **Story: As a customer, I want to pay online so that I don’t need cash.**

Scenario 1: Success

Given I have items in my cart

When I pay with a valid bKash or card account

Then my order is confirmed and I get a receipt

Scenario 2: Failure

Given I have items in my cart

When my payment fails

Then I see an error and my cart is kept

What Is a Product Backlog?

The single, ordered list of everything the product needs. The team always works from the top.

Prioritized

Most important stories at the top

Living

Changes as the team and customer learn more

Owned

The Product Owner decides the order

Detailed at the Top

Top stories are clear; lower ones can stay rough

MoSCoW Prioritization

MUST have

The product fails without it

e.g. place an order

SHOULD have

Important, but the product works without it

e.g. track order status

COULD have

Nice extra, if time allows

e.g. rate a dish

WON’T have (this time)

Agreed to leave out of this version

e.g. loyalty points

Rule of thumb: if almost everything is a Must, nothing is really prioritized.

Worked Example: A Sample Backlog

ID	User Story (Online Food Ordering)	Priority
US1	As a customer, I want to browse the menu so that I can choose food	**Must**
US2	As a customer, I want to add items to a cart so that I can order several dishes	**Must**
US3	As a customer, I want to pay online so that I don’t need cash	**Must**
US4	As a customer, I want to track my order so that I know when it arrives	**Should**
US5	As a branch manager, I want to update menu items so that the menu stays correct	**Should**
US6	As a customer, I want to rate dishes so that others can choose well	**Could**
US7	As a customer, I want loyalty points so that I get discounts	**Won’t (v1)**

Task: Build Your Product Backlog

1. Pick 3–4 features from your charter's scope
2. Write 10–15 user stories: As a … I want … so that …
3. Add acceptance criteria (Given / When / Then) to your top 5
4. Label every story Must, Should, Could or Won't
5. List 3–5 non-functional requirements
6. Put every story in the Backlog column of your board

> **One member acts as Product Owner and makes the final call on priorities.**

Setting Up Your Board

Backlog	To Do	In Progress	Done

Card title: US3 – Pay online

Description: As a customer, I want to pay online so that I don't need cash.

Checklist: acceptance criteria

Label: Must

Today: all stories go in Backlog, Musts at the top.

Lab 3: sprint stories move to To Do.

Use labels or colours for Must, Should, Could, Won't.

Deliverable: Due Before Lab 3

- 10–15 user stories on your board, each with a MoSCoW label
- Acceptance criteria for your top 5 stories
- 3–5 measurable non-functional requirements
- Backlog saved as /docs/backlog.md in your team repo
- Peer-review feedback applied

> **Your backlog is the starting point for sprint planning in Lab 3.**

Coming Up: Lab 3

Git & GitHub Workflow and Sprint Planning

- Clone, branch, commit and pull request
- Resolve a merge conflict as a team
- Estimate your stories with planning poker
- Plan your first sprint from your Must stories

> **Bring your backlog: it is the input for sprint planning.**

Key Takeaways

- Functional = what it does; non-functional = how well
- User story: As a … I want … so that …
- Good stories pass INVEST and have acceptance criteria
- MoSCoW keeps the backlog honestly prioritized

Questions?